



## Agile Development II — XP, Kanban and the Limits of Agile

*The engineering half Scrum leaves out, the flow it cannot see, and the line above which none of it applies*

Session 6 of 28 · Friday 25 September 2026 · 10:00 – 11:20 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

### SESSION 6 OF 28

## Agile Development II — XP, Kanban and the Limits of Agile

Friday, 25 September 2026 · 10:00 – 11:20 AM · Unit 1 · Foundations and Process

### LEARNING OBJECTIVES

- ✓ Explain why a team that runs Scrum and nothing else slows down after a few sprints.
- ✓ Name the XP practices, say which ones this team keeps, and state the two rules that replace the rest.
- ✓ Use the describe–build–check loop to build a real SANTE-BASSAM function with an AI.
- ✓ Name what Scrum cannot see inside a sprint, and define Kanban as the remedy.
- ✓ Use WIP, cycle time, throughput and Little’s Law to find the bottleneck in a flow log.
- ✓ Identify the parts of SANTE-BASSAM that must not be run in sprints, and say why.

### FROM SESSION 5

Scrum gave the team a date — Friday 16 October — when the clerk first touches the software, and a new one every fortnight: stories with acceptance criteria, points and velocity, three accountabilities, three artifacts, five events, a Definition of Done.

### Revised 28 September.

The team runs Scrum (Session 5) with a light Kanban board on top. It does *not* adopt Extreme Programming: XP is described in §2 because every software engineer should know it, and §2.1–2.2 give the lighter practice this team uses instead — AI as a helper, held to two rules. There is no build pipeline, and no test has to be written to fail first.

### TEAM PROJECT · 0

### first 15 minutes of class

### Sprint 1 tasks — and what each one waits on

Applying Session 5 (Scrum) and Sessions 3–4 (plan-driven ordering). You use only the tools you already have. Kanban has not been introduced yet — that is deliberate.

#### Pair A

tasks for **S-04** — register a patient with the paper-card fields (5 pts)

#### Pair B

tasks for **S-06** — search results in under two seconds (3 pts)

#### Pair C

the dependencies: for every task of S-04, S-05 and S-06, what does it *wait on*?

1. **Six minutes in pairs.** A and B: each task is a verb and an object, one day or less, one named owner, and the test that proves it finished — the shape of S-05's six tasks in Session 5 §5.3. C: start from S-05's tasks; each one waits on *another task*, a *person* (the clerk, a reviewer) or *nothing*.
2. **Five minutes on the whiteboard.** A and B read out their tasks; C draws an arrow from each task to what it waits on. Every one of the six leaves owning at least one task.
3. **Four minutes: the record.** Using only Scrum's artifacts and events, answer the three questions below in the progression log. Then leave them alone — you reopen them in §5.

#### Before Kanban — the team's sealed answers

- P1** Which task will the most work end up waiting on? Name it.
- P2** On which day of the sprint will the burndown first show a problem — and will it show *what* the problem is?
- P3** A task is blocked on Tuesday. When does the team find out, and what does it do?

**Output:** the sprint 1 task list with owners, the dependency arrows, and P1–P3 — all in the progression log. If your name is not against a task, you have not started sprint 1.

**Next session:** the first 50 minutes become a work block — two groups of three turn the logged tasks into step-by-step plans.

## 1 · What Scrum leaves out

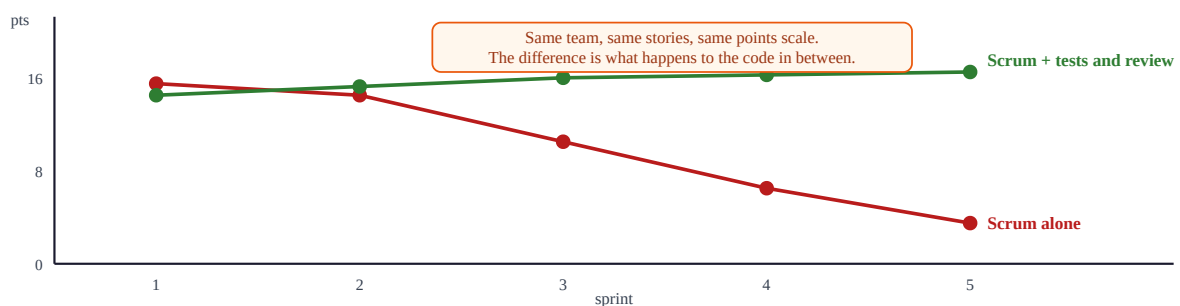
Scrum schedules *when* the clerk sees the software. None of its five events mentions a test, a design or a line of code, and none of its artifacts shows where a started item is sitting. That leaves two gaps, and today fills one each.

### Gap 1 · code that can absorb change

Nothing in Scrum keeps the code cheap to change in sprint 4. **XP's answer; and this team's lighter one** (§2).

### Gap 2 · work that moves once started

Nothing in Scrum shows what a started item is waiting for. **Kanban fills it** (§3).

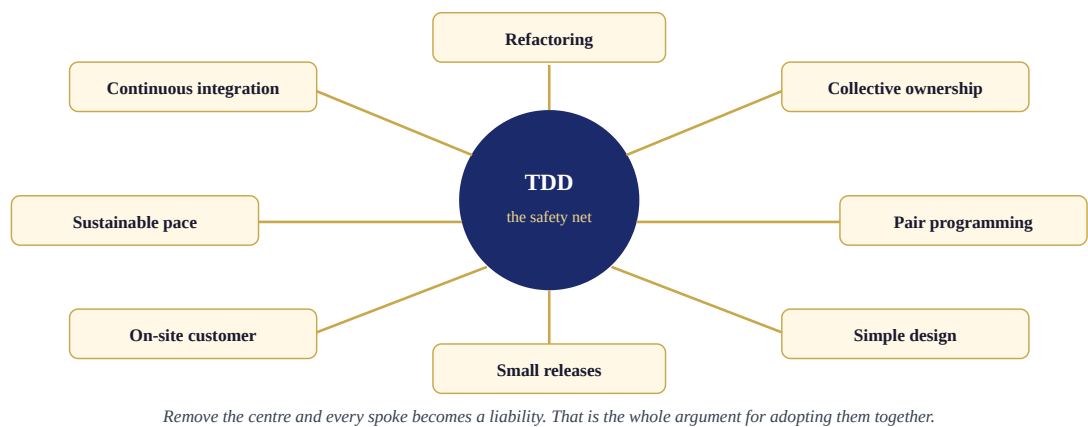


**Figure 1.** Sprints 1–2 are fast because the code is new. From sprint 3 every change lands in untested code, so each story costs more than the last. The team concludes agile does not work; what failed was the half Scrum never claimed to provide.

## 2 · The engineering half: what XP says, and what this team keeps

Extreme Programming (XP) is the classic answer to Gap 1. It takes practices everyone agrees are good and pushes them to the limit: if review is good, review every line as it is typed; if testing is good, test before the code exists. It is also **demanding** — pairing all day, a test before every line, a customer on site — and not feasible for six students carrying four other courses. **This team does not adopt XP.** It keeps a few of its ideas in a lighter form, and replaces the rest with something XP never had: an AI that writes code on request.

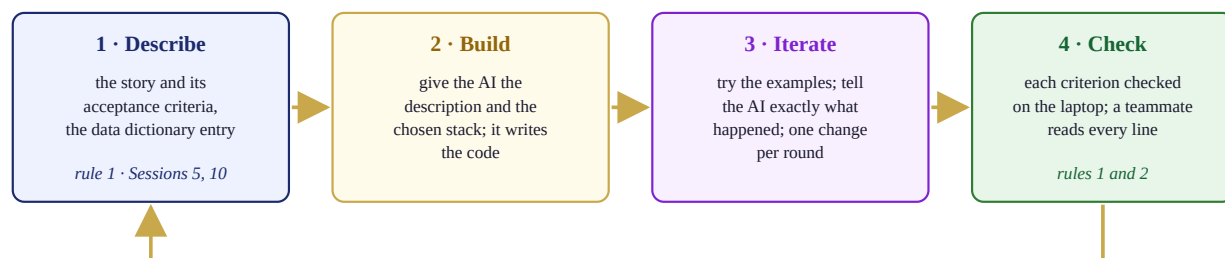
| XP practice             | What it is                                                             | This team                                                                                                                                |
|-------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Test-driven development | Failing test first, least code to pass, then tidy with the test green. | <b>not adopted</b> Instead, each story's acceptance criteria describe the behaviour precisely enough to build from and to check against. |
| Continuous integration  | Merge within hours; every push runs the tests.                         | <b>not adopted</b> No build pipeline. Before a review, each acceptance criterion is checked by hand on the target device.                |
| Pair programming        | Two people, one keyboard, all day.                                     | <b>not adopted</b> Replaced by rule 2: a teammate reads AI-generated code before it merges.                                              |
| Collective ownership    | Anyone fixes a defect, anywhere.                                       | <b>kept</b>                                                                                                                              |
| Refactoring             | Change structure, not behaviour, continuously.                         | <b>not required</b> If you ask the AI to restructure code, re-check the story's acceptance criteria afterwards.                          |
| Simple design           | Build only what today's stories need.                                  | <b>kept</b> No user roles until a story needs them.                                                                                      |
| Small releases          | Something usable every sprint.                                         | <b>kept</b> Every sprint review, every second Friday.                                                                                    |
| On-site customer        | Someone who can answer questions at any time.                          | <b>replaced</b> The instructor stands in; the Product Owner may ask one question a day.                                                  |
| Sustainable pace        | Plan only what you can keep up.                                        | <b>kept</b> Plan to measured velocity, once sprint 1 has produced one.                                                                   |



**Figure 2.** XP's own argument: each practice on the rim is risky alone and safe with tests at the centre. This team keeps the spirit of the centre — knowing what “working” means before anything is built — as precise acceptance criteria, because code an AI wrote is the code nobody on the team has read yet.

## 2.1 Building with AI: describe, build, check

Andrew Ng's short course *Build Your Own App in 30 Minutes* teaches a loop anyone can follow: **describe** the app in words, let the AI **build** it, **iterate** by conversation, then **test** it and share it. The course is written for beginners with no programming experience. On SANTE-BASSAM the loop is the same — what changes is that every step uses an artifact you already have.



next acceptance criterion — or, if the review shows the story was wrong, a new conversation with the clerk  
Ng's loop is describe → build → iterate → test. Here the description is the story, and a human reads what the AI wrote.

**Figure 3.** The loop on one story. The story and its acceptance criteria are the description the AI needs — and the same criteria tell you whether what came back does the job.

- 1 Describe.** S-05 needs `rank(term, patient)`, a score from 0 to 100. Give the AI S-05's criterion AC2 — *a name misspelled by one letter is still found in the first five results* — with two examples ("Kofi Modeste" must find Kofi Modeste) and the Patient fields.
- 2 Build.** Ask for the simplest version that meets AC2, in the team's stack.
- 3 Iterate.** Try the examples. If Kofi is not found, tell the AI exactly what happened — "I searched Kofi Modeste and Kofi was 9th" — one change per round, never "make it better".
- 4 Check.** On the desk laptop, check AC1 to AC3 with the stand-in clerk and write down the result. A teammate who did not prompt the code reads every line and can explain it. Only then does it merge.

### THE SKILL AI DOES NOT REPLACE

With a frontier model, a precise description is now close to the only step between an idea and working software — the observation behind Andrew Ng's app-building course, in which you describe the app in words and the AI builds it. The skill is not new: it is what a good specification has always been. What is new is that it now decides almost everything. **If someone could not build it from your description, neither can the AI.**

## 2.2 The two rules — this team's whole engineering discipline

### AI IS ALLOWED FOR DESIGN, TESTS AND CODE — UNDER TWO RULES

**Rule 1 · Describe it precisely, then check against the description.** Every story's acceptance criteria are precise enough that someone — or an AI — could build it from them, and each one is checked on the target device before the review.

**Rule 2 · Nothing generated merges unread.** A teammate who did not prompt the code reads it line by line and can explain it. Whoever accepts it owns it: "the model wrote it" is not a defence.

### The failure to expect.

Teams keep the AI and drop the rules at 23:00 the night before a review. On SANTE-BASSAM the code most likely to be merged unread is the sync path — and a defect there is a lost health record.

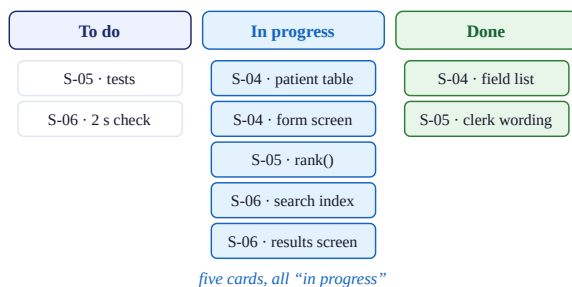
## 3 · Kanban: managing the flow Scrum cannot see

### 3.1 Why — four things Scrum does not show you

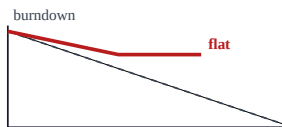
Scrum controls what *enters* a sprint. Once a sprint starts, everything between “started” and “done” is one column called *In progress*. Here is what that hides, on your own sprint 1:

- 1 Waiting looks like working.** A story in *In progress* may be coded and sitting three days for a reviewer. The board cannot tell the two apart.
- 2 Dependencies travel by word of mouth.** S-06’s search index waits on S-04’s patient table. No Scrum artifact records “waits on”; the team hears about it at the next daily scrum — if someone says it.
- 3 Nothing limits starting.** Velocity caps how much enters the sprint, not how much is open at once. Six people start six things; on review Friday five are 80% done — and 80% done is not done.
- 4 Late signal, no diagnosis.** The burndown measures work *remaining*. It goes flat and says “late”; it never says *where* the work is stuck.

Scrum task board — day 7 of 10

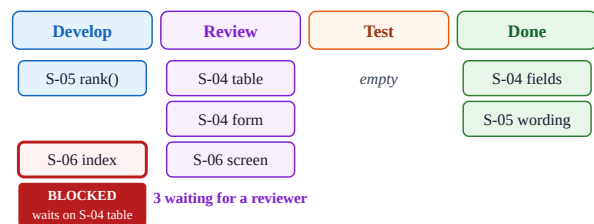


burndown



Flat since day 5.  
**It says late.**  
**It cannot say why.**

The same work on a flow board



Where the five “in progress” items really are



S-06 index waits on S-04 table, which waits on a reviewer.

**Fix: stop starting; two people go and review.**

Nobody on this board is slow.

**Figure 4.** One Wednesday, two boards. The left one is correct Scrum and shows five busy cards and a late burndown. The right one shows the same five cards by the state they are actually in — and the diagnosis is on the board.

### 3.2 What Kanban is

#### DEFINITION

**Kanban** (Japanese: “signboard”) manages work as a **continuous flow of single items** rather than a batch per timebox. It rests on three practices:

#### Visualise

one column for every state work really passes through — including waiting states like Review — and a red tag on every blocked item saying what it waits on.

#### Limit WIP

a maximum on each column. When a column is full, nobody pulls into it; you go and help empty it.

#### Measure flow

record the day each item enters and leaves each column, then fix the column where items wait longest.

| What Scrum does not show (§3.1)   | Kanban's remedy                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------|
| 1 · Waiting looks like working    | Waiting states become columns: <i>coded</i> , <i>waiting for review</i> is visible. |
| 2 · Dependencies by word of mouth | Blocked tag + “waits on” on the card — seen the same hour, not the next stand-up.   |
| 3 · Nothing limits starting       | WIP limits: starting is refused until something finishes.                           |
| 4 · Late signal, no diagnosis     | Time per column: the slowest column is the diagnosis.                               |

#### What Kanban does not do.

It does not *schedule* dependencies — ordering tasks around a dependency network is the plan-driven tool from Sessions 3–4. Kanban makes a blocked item impossible to miss, and cheap to act on the same day.

### 3.3 Measuring flow — three numbers and one law

#### DEFINITIONS

**Work in progress (WIP)** — items started and not finished.

**Cycle time** — days from an item entering the first active column to reaching Done.

**Throughput** — items finished per unit of time, for the whole team.

$$\text{average WIP} = \text{throughput} \times \text{average cycle time}$$

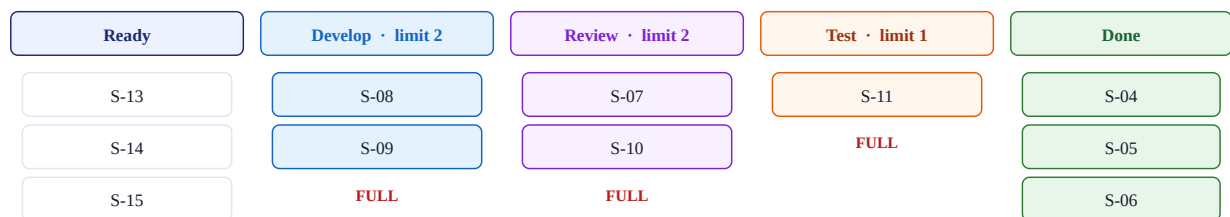
Little's Law,  $L = \lambda W$  — holds for any stable system, whatever happens inside it

Rearranged: **cycle time** = **WIP** ÷ **throughput**. Finishing 1.5 stories a week with 6 started means each takes 4 weeks, however hard anyone works.

**The only way to finish sooner is to start less.**

#### WHY THIS MATTERS MORE TO A TEAM OF SIX

Six people want six things open so nobody looks idle. A Develop limit of two says: **help finish, don't start** — two people on one story, one prompting the AI and one reading what comes back. The WIP limit and rule 2 are one habit seen from two sides.



#### Nothing can move. Read the board before you read this.

A developer finishes S-08. Review is full, so S-08 cannot advance. She wants to pull S-13 from Ready — but Develop would then hold 2 again the moment S-08 leaves, and the queue in front of Review grows. **The correct move is to stop coding and go and review S-07.**

*This is the whole idea: a WIP limit converts individual busyness into system throughput, by making waiting visible and starting expensive.*

**Figure 5.** The SANTE-BASSAM board in sprint 2, now with limits. Every active column is full, so finishing S-08 does not license starting S-13. Without limits the same jam is invisible: everyone busy, nothing finishing.

3.4 One story worked in full

COUNTING CONVENTION — EVERYONE USES THIS ONE

- Working days only.** Mon wk5 = day 1, Fri wk5 = day 5, Mon wk6 = day 6 ... Fri wk8 = day 20.
- An item leaves one column on the same day it enters the next, so that day is counted once — in the first column.
- Develop** = left – entered + 1 · **Review** and **Test** = left – entered.
- Cycle time** = Done – entered Develop + 1 = Develop + Review + Test.

| S-04       | Entered Develop                    | Left Develop | Left Review        | Done             |
|------------|------------------------------------|--------------|--------------------|------------------|
| as dates   | Mon wk5                            | Wed wk5      | Mon wk6            | Tue wk6          |
| as days    | 1                                  | 3            | 6                  | 7                |
| per column | Develop = 3 – 1 + 1 = 3            |              | Review = 6 – 3 = 3 | Test = 7 – 6 = 1 |
|            | Cycle time = 7 – 1 + 1 = 3 + 3 + 1 |              |                    | 7 days           |

Three days coded, three days waiting for review, one day tested. Nobody was lazy.

3.5 Kanban or Scrum?

|                      | Scrum                                             | Kanban                                                    |
|----------------------|---------------------------------------------------|-----------------------------------------------------------|
| Unit of planning     | The sprint — a fixed timebox                      | The single item — no timebox                              |
| Controls overload by | Committing only to measured velocity              | Refusing to start — WIP limits                            |
| Key metric           | Velocity (points per sprint)                      | Cycle time and throughput                                 |
| Suits                | Work batched behind a review date with a customer | A stream of arrivals — support, bug reports after go-live |
| Costs you            | The overhead of five events                       | The deadline: no review Friday, nobody prepares           |

Not rivals.

Many teams keep the sprint and put WIP limits on its board (“Scrumban”): the sprint supplies the date, the limits stop six stories being half-finished on the morning of it.

4 · Where agile does not fit

Agile is rational when feedback is cheap (Session 5). When it is not, recognise it — do not apologise for it.

No accessible customer

No answers between reviews: the Product Owner is guessing, and everything inherits the guess.

Feedback is expensive

Implanted devices, satellites — nothing you can half-deploy and ask about.

### Regulatory evidence dominates

Certification needs traceable specifications written in advance. Plan for them; sprints will not produce them.

### Fixed price, fixed scope

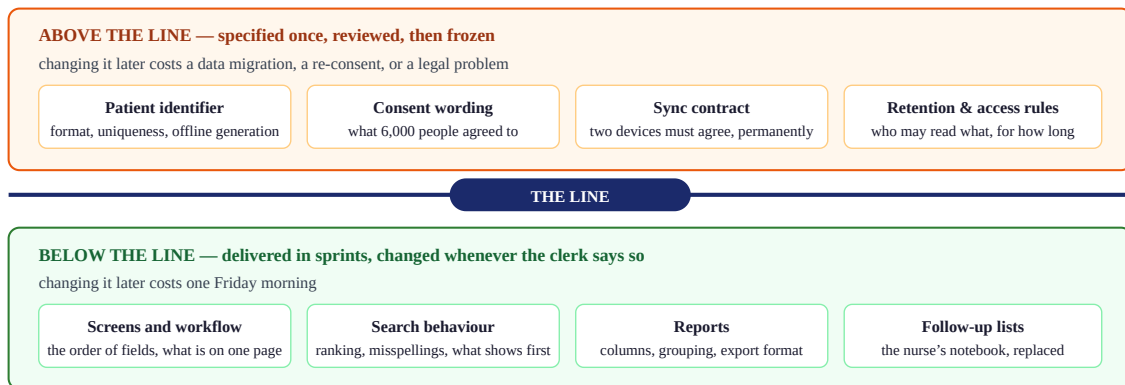
The contract forbids “responding to change”, in writing.

### Interfaces others depend on

Iterate behind an interface, never on it once a second party has built against it.

## APPLIED TO SANTE-BASSAM

*The line your Project Plan has to draw — one project, two methods*



*Same project. Same team. Same fortnight. Two methods, because the cost of being wrong is different on each side.*

**Figure 6.** *The Project Plan does not choose between plan-driven and agile. It draws this line and defends where it put it.*

## SESSION 6 TAKEAWAY

Scrum schedules the feedback. The code absorbs it only if each story is described precisely, checked against that description, and a teammate reads what the AI wrote — XP's centre, kept in a lighter form. Kanban shows where started work is waiting and refuses to start more until it moves. None of it applies above the line — the identifier, the consent, the sync contract.

## 5 · In-class exercise · where does the time go?

Twelve minutes. The flow log is a previous SANTE-BASSAM team's first four weeks. Use the convention in §3.4; S-04 is worked there.



| Story                    | Entered Develop | Left Develop | Left Review       | Done (test passed)              |
|--------------------------|-----------------|--------------|-------------------|---------------------------------|
| S-04                     | Mon wk5         | Wed wk5      | Mon wk6           | Tue wk6 — <i>worked in §3.4</i> |
| S-05                     | Mon wk5         | Fri wk5      | Thu wk6           | Fri wk6                         |
| S-06                     | Thu wk5         | Fri wk5      | Thu wk6           | Fri wk6                         |
| S-07                     | Mon wk7         | Wed wk7      | Tue wk8           | Wed wk8                         |
| S-08                     | Tue wk7         | Thu wk7      | Wed wk8           | Thu wk8                         |
| S-09                     | Thu wk7         | Fri wk7      | Thu wk8           | Fri wk8                         |
| S-15 <i>pulled early</i> | Mon wk8         | Tue wk8      | — still in Review | not done                        |
| S-16 <i>pulled early</i> | Tue wk8         | Wed wk8      | — still in Review | not done                        |

One story each, then the diagnosis (12 minutes)

- 1. Four minutes.** One completed story each (whoever takes S-04 checks it, then helps a neighbour): three column times and the cycle time.
- 2. Four minutes, together at the board.** Average cycle time; throughput in stories per week over four weeks; average WIP by Little's Law. Leave S-15 and S-16 out — they never finished.
- 3. Four minutes: open the sealed answers.** Which column holds work longest — give the number of days. Now read P1–P3 from the start of class aloud. For each, write one line in the log: *what the flow board shows that Scrum did not*. Last line: S-15 and S-16 were started in week 8 while three stories were unfinished — what did that do to everything in flight?

|                                      |                                                  |                                |                              |                      |
|--------------------------------------|--------------------------------------------------|--------------------------------|------------------------------|----------------------|
| story                                | develop                                          | review                         | test                         | cycle time           |
| S-04                                 | 3                                                | 3                              | 1                            | 7                    |
| S-05                                 | <input type="text"/>                             | <input type="text"/>           | <input type="text"/>         | <input type="text"/> |
| ...                                  |                                                  |                                |                              |                      |
| average cycle time                   | = <input type="text"/>                           | working days                   |                              |                      |
| throughput                           | = <input type="text"/>                           | stories / <input type="text"/> | weeks = <input type="text"/> | per week             |
| average WIP                          | = throughput × cycle time = <input type="text"/> | stories                        |                              |                      |
| P1 we predicted <input type="text"/> | the board shows <input type="text"/>             |                                |                              |                      |
| P2 we predicted <input type="text"/> | the board shows <input type="text"/>             |                                |                              |                      |
| P3 we predicted <input type="text"/> | the board shows <input type="text"/>             |                                |                              |                      |

6 · Homework · individual · due before Session 7 (Tuesday 29 September)

About forty-five minutes, nothing hard. Every question continues something from class.

Download `CSC4305_HW06_Template.md` from `CSC_4305_SWE-Assignments`, type between the `<!-- ANSWER -->` markers, rename it `CSC4305_HW06_FirstName_LastName.md` and commit it to `/homework/S06/` in your own private repository.

Markdown only. Drafting in Word is fine — convert at [word2md.com](https://word2md.com) and commit the `.md`.

- Q1

**Two stories, arithmetic visible.** The one you took in class and one other. Day numbers, three column times and cycle time, in the shape of the S-04 table. Then one sentence: which of the two was more *waited on* than *worked on*, and how you can tell.

A reader must be able to find your mistake.

- Q2 Little's Law, on your own.** From the team figures, compute the average WIP. Then two sentences: if each of the six started one more story tomorrow, what happens to the cycle time of everything in flight — and to throughput?  
The second half surprises people. Say it plainly.
- Q3 Before and after Kanban.** Copy your team's P1–P3 from the progression log. For each, one sentence: what the flow board or flow log showed, and which Scrum artifact or event could not have shown it. Finish with one sentence on what Kanban would *not* have fixed.
- Q4 The two rules, under pressure.** It is 23:00 before a review, and an AI has just written the sync-retry code for S-09. In three or four sentences: what goes wrong for a patient record if the team skips rule 1? What if it skips rule 2?  
Name the consequence for a patient record, not for “quality”.
- Q5 Draw the line yourself.** One thing SANTE-BASSAM will need that is not in Figure 6. Which side of the line, and why — two sentences. The test: what does changing your mind cost in week 12?
- Q6 Your opinion, three sentences.** Should the team switch to Kanban for sprints 3–5, given that the review with the nurse is what makes the sprint boundary worth its overhead? State one assumption, one trade-off you accepted, one question the data could not answer.  
There is no correct answer. There are unsupported ones.
- Q7 Reading.** Sommerville, *Software Engineering* 10th ed., §3.3–3.4. Optional: Beck, *Extreme Programming Explained*, ch. 1–3; Anderson, *Kanban*, ch. 1–2.

## Key terms

Extreme Programming

test-driven development

describe-build-check

the two rules

continuous integration

pair programming

collective ownership

refactoring

simple design

small releases

sustainable pace

Kanban

blocked item

work in progress

WIP limit

cycle time

throughput

Little's Law

Scrumban

the line

## Readings and references

- Sommerville, I. *Software Engineering*, 10th ed. — Chapter 3, §3.3–3.4.
- Beck, K. *Extreme Programming Explained: Embrace Change*, 2nd ed., 2004.
- Ng, A. *Build Your Own App In Just 30 Minutes! Full Course* — [youtube.com/watch?v=ff3j4olCUig](https://youtube.com/watch?v=ff3j4olCUig).
- Anderson, D. *Kanban: Successful Evolutionary Change for Your Technology Business*, 2010.
- Little, J. D. C. “A Proof for the Queuing Formula  $L = \lambda W$ ”, *Operations Research*, 1961.
- Pressman, R. & Maxim, B. *Software Engineering: A Practitioner's Approach* — Chapter 3.